

NEXUS AI

Testing, Security & Release Document

N63–N72 • Testing • AI Evaluation • RAG • Agent Safety • Security • Performance • Offline • Packaging •
Release

Version: 0.1 **Status:** Pre-release Quality & Launch Foundation **Target:** Windows Local-first Application

Purpose: Define how NEXUS AI is validated before release, including functional correctness, AI quality, RAG grounding, agent safety, local security, performance, offline operation, Windows packaging and final MVP release approval.

BUILD → TEST → MEASURE → FIX → REGRESS → HARDEN → PACKAGE → RELEASE → MONITOR

1. Quality Engineering Vision

Testing for NEXUS is not limited to whether screens work. The release must prove that local AI produces useful results, RAG answers are grounded, agents cannot bypass permissions, the application remains usable under local hardware constraints, and the core product works without internet access.

Quality Principles

- Test deterministic software behavior separately from probabilistic AI behavior.
- Automate repeatable tests and reserve human review for quality judgments that require it.
- Security controls are tested outside the model; prompts are never treated as a security boundary.
- AI changes require regression evaluation.
- Destructive/high-risk agent operations require negative testing.
- Offline mode must be tested with network access actually disabled.
- Performance tests use the target hardware class, not only developer machines.
- Every release candidate must pass a defined release gate.

2. N63 — Testing Strategy

Test Layer	Scope	Examples
Unit	Individual functions/components	Parsing, routing, permissions, validators
Integration	Subsystem contracts	Ollama, DB, RAG, tools, agent runtime
API	Backend contracts	Auth/session, chat, knowledge, agent, model APIs
UI	User interactions	Navigation, chat, projects, settings
E2E	Complete workflows	Chat, RAG, coding agent, backup/recovery
AI Quality	Model behavior	Instruction following, coding, reasoning
Security	Attack/abuse resistance	Injection, path traversal, command abuse
Performance	Resource/latency	RAM, VRAM, CPU, GPU, disk, response time
Offline	No-network operation	Local chat, RAG, agents, project workflows
Release	Install/update/recovery	Installer, upgrade, uninstall, rollback

Testing Pyramid

Unit tests form the broad base, followed by integration/API tests, then UI/E2E tests. AI evaluation, security and performance suites run as specialized quality gates across the pyramid.

3. N64 — AI Evaluation & Benchmark Test Plan

AI quality is measured using repeatable benchmark sets and task-specific metrics. A model passing a generic chat test does not prove that it is suitable for coding, RAG or agent planning.

Benchmark	Measure
General instruction	Instruction adherence, relevance, clarity
Reasoning	Correctness and reasoning task success
Coding	Tests passed, correctness, code quality
Debugging	Fault localization and repair success
Structured output	Schema validity and field correctness
Tool planning	Correct tool selection and arguments
Long context	Relevant information retention
Safety	Policy compliance and unsafe-action resistance

Benchmark	Measure
Latency	TTFT, throughput, end-to-end response time
Resource	RAM/VRAM/GPU/CPU usage

AI Regression Rule

Every model, prompt, routing, context or verifier change should be evaluated against the previous approved baseline. Significant quality regressions block release until accepted or explicitly waived.

4. N65 — RAG Evaluation Test Plan

RAG testing separates retrieval quality from answer quality.

Test Area	Metrics / Checks
Ingestion	Text extraction accuracy, page/section preservation
Chunking	Boundary quality, metadata correctness
Retrieval	Recall@K, Precision@K, MRR/rank quality
Hybrid search	Semantic + lexical contribution
Reranking	Relevant evidence promoted into top results
Grounding	Claims supported by retrieved evidence
Citations	Correct document/page/section references
Versioning	Correct active document version
Access control	Unauthorized collections never retrieved
No evidence	System states insufficient evidence

Golden Dataset

Maintain a small labeled set of real NEXUS documents and questions with expected source chunks. Run it whenever embedding, chunking, retrieval, reranking or citation logic changes.

5. N66 — Agent Safety Test Plan

Agent testing focuses on whether the model can propose actions without gaining unauthorized execution power.

Scenario	Expected Result
Read allowed file	Succeeds within workspace.
Read outside workspace	Blocked.
Path traversal	Blocked.
Delete project file	Requires configured approval or is blocked.
Destructive terminal command	Blocked/approval required.
Unknown command	Policy rejects or asks for approval.
Prompt injection in file	Treated as untrusted content.
Tool output injection	Cannot alter permissions.
Repeated failed action	Loop budget stops execution.
Task cancellation	Agent stops safely.
Timeout	Process/task terminates according to policy.
Unauthorized network action	Blocked in offline policy.

Agent Safety Gate

No release if an agent can bypass filesystem scope, command restrictions, approval controls, task budgets or cancellation.

6. N67 — Security Test Plan

NEXUS security testing covers the desktop application, local services, storage, permissions and AI/tool boundaries.

Security Domain	Tests
Filesystem	Traversal, symlink escape, unauthorized paths, unsafe writes
Terminal	Command injection, argument manipulation, privilege attempts
Application API	Invalid input, malformed requests, unauthorized operations
Database	Injection, access boundary, corruption/recovery
Secrets	Log leakage, config exposure, export leakage
RAG	Prompt injection, unauthorized retrieval, malicious documents
Agent	Tool abuse, permission escalation, loop abuse
Network	Unexpected outbound calls, offline enforcement
Updates	Package integrity and downgrade/rollback behavior
Local data	File permissions and sensitive data exposure

Security Principle

Test the enforcement layer directly. A model refusal is not considered a successful security control if the underlying tool gateway would still execute the action.

7. Security Abuse Cases

- Agent attempts to access C:\ outside the configured workspace.
- Document contains instructions pretending to be system policy.
- Terminal output contains instructions to execute another command.
- User edits tool arguments after model generation.
- Malicious filename or path attempts traversal.
- Imported document attempts resource exhaustion.
- User disables a security control and then performs a high-risk action.
- Backup/export accidentally includes secrets.

8. N68 — Performance Test Plan

Performance testing targets the real local constraints of NEXUS: GPU memory, system RAM, CPU load, model loading time, storage speed and concurrent background tasks.

Metric	Target / Measurement
Application startup	Time to interactive after launch
Model load	Time from request to model ready
TTFT	Time to first generated token
Throughput	Tokens per second
RAG search	Query-to-results latency
Agent task	End-to-end completion time
UI responsiveness	Interaction latency during AI workloads
RAM	Idle and peak usage
VRAM	Model inference peak
CPU/GPU	Utilization during workloads
Disk	Index/cache/model growth

Performance Scenarios

Cold model load • warm model • long-context prompt • RAG search • coding task • multi-step agent • simultaneous UI + inference • background ingestion • low-disk condition.

Resource Protection

NEXUS should detect high resource pressure and reduce concurrency, select smaller models, queue tasks or stop nonessential background work rather than allowing the entire application to become unstable.

9. N69 — Offline Capability Test Plan

Offline mode is a product requirement, not merely an error state. Testing must disconnect network access rather than simply assuming it is unavailable.

Workflow	Offline Requirement
Launch	Application starts normally.
Chat	Local Ollama model responds.
Model management	Installed models remain usable.
RAG	Local documents can be searched.
Projects	Files and project data work normally.
Agent	Approved local tools work.
Terminal/tests	Local commands work according to policy.
Memory	Local memory persists and retrieves.
Settings	Privacy/offline controls remain available.
Cloud-only feature	Clearly unavailable rather than silently failing.

Offline Verification

Run release tests with network interfaces disabled/firewalled and inspect application logs/network activity where practical. No mandatory cloud request should be required for the core MVP workflow.

10. N70 — Packaging & Installation Plan

NEXUS is packaged as a Windows desktop application with an installer that configures the application without forcing users to manually assemble the system.

Installation Area	Requirement
Installer	Signed/verified release package where signing infrastructure is available.
Application files	Installed into appropriate user/system location.
User data	Stored separately from application binaries.
Database	Created/migrated automatically.
Ollama	Detect existing installation; provide clear setup guidance if missing.
Models	Do not silently download huge models without user consent.
First launch	Runtime checks and setup wizard.
Uninstall	Clearly state whether user data is retained or removed.
Upgrade	Preserve compatible user data and configuration.

Installation Test Matrix

Fresh install • existing Ollama • missing Ollama • existing models • no models • upgrade from previous version • corrupted config • insufficient disk • uninstall/reinstall • offline installation.

11. Release Architecture

Release artifacts should contain the desktop application, version metadata, release notes and integrity information. User data should remain outside the application package so updates do not overwrite it.

12. N71 — Release Management Plan

Release Type	Purpose
Development build	Rapid internal iteration.
Alpha	Early integrated functionality; unstable.
Beta/RC	Feature-complete candidate for broad testing.
MVP Release	Approved stable public/user release.
Hotfix	Critical correction with minimal scope.

Versioning

Use semantic-style versioning: MAJOR for incompatible product changes, MINOR for backward-compatible features, PATCH for fixes. Pre-release labels may identify alpha/beta/release candidates.

Release Flow

Code Freeze → Build → Automated Tests → AI/RAG Evaluation → Security Tests → Performance → Offline Tests → Installer Validation → Release Candidate → Final Approval → Publish → Post-release Verification.

13. Release Candidate Gate

- All P0 functional tests pass.
- No unresolved critical security issue.
- AI benchmark is within approved baseline.
- RAG retrieval and citation quality meet thresholds.
- Agent safety suite passes.
- Offline core workflows pass.
- Performance remains within hardware targets.
- Fresh installation succeeds.
- Upgrade path succeeds.
- Backup/recovery has been validated.
- Release notes and known limitations are complete.

14. N72 — MVP Release Checklist

Area	Gate
Application	Build launches and core screens work.
AI	Local models discovered and inference works.
Chat	Streaming, cancellation and persistence work.
Projects	Workspace isolation and persistence work.
RAG	Import → search → grounded answer → citation works.
Agents	Planning, tools, permissions and cancellation work.
Security	Critical abuse cases blocked.
Performance	Target hardware remains stable.
Offline	Core MVP operates without internet.
Data	Migration, backup and recovery validated.

Area	Gate
Installer	Fresh install/upgrade/uninstall validated.
Documentation	User setup, privacy and known limitations documented.

15. Defect Severity

Severity	Definition	Release Impact
P0 Critical	Data loss, security bypass, uncontrolled execution, unusable launch	Release blocked
P1 High	Major workflow broken, severe AI/RAG regression	Normally blocked
P2 Medium	Workaround exists; non-critical feature issue	May release with approval
P3 Low	Cosmetic/minor usability issue	Can defer

16. Test Environment Matrix

Environment	Purpose
Developer machine	Fast unit/integration validation.
Clean Windows VM/device	Installer and clean-environment testing.
Target hardware	AI performance and resource validation.
Offline environment	Network-disabled acceptance.
Low-resource scenario	RAM/VRAM pressure behavior.
Upgrade scenario	Migration and compatibility.

17. Test Data Management

Use synthetic and controlled test documents for automated testing. Real user data should not be required for CI or shared test environments. AI benchmark datasets must be versioned so score changes remain comparable.

Sensitive Data Rule

Do not commit personal documents, secrets, API keys or private conversations into test repositories. Test fixtures should be sanitized and intentionally constructed.

18. AI Test Determinism

Local LLM output can vary. Evaluation therefore uses task-specific scoring rather than exact text matching wherever possible. Deterministic checks remain preferred for JSON schemas, tool arguments, file diffs, tests and citations.

Record model ID/tag, prompt version, benchmark version, runtime settings and hardware for reproducible comparisons.

19. Continuous Regression

A release pipeline should run a lightweight test suite on every change and a broader AI/security/performance suite for release candidates.

Trigger	Suite
Every code change	Unit + lint/type checks + focused integration
Feature merge	Integration + API + relevant UI tests
Model/prompt change	AI benchmark + routing + safety regression
RAG change	RAG golden dataset + citation tests
Agent/tool change	Agent safety + permission tests
Release candidate	Full suite + performance + offline + packaging

20. Monitoring After Release

Release quality continues after launch. Track crashes, model errors, failed ingestion, agent failures, tool denials, performance regressions and user-reported issues while respecting local privacy.

The default product should keep telemetry local unless the user explicitly enables an external reporting mechanism.

21. Rollback & Incident Response

If a release causes severe problems: stop further distribution → identify affected version → communicate issue → provide rollback/fix → validate recovery → publish corrected release.

Data Safety

Application rollback must not automatically roll back user data unless a documented migration/recovery process exists. Database migrations should be backward-aware where practical.

22. MVP Release Readiness Scorecard

Gate	Status Required
Functional tests	PASS
AI evaluation	PASS / approved variance
RAG evaluation	PASS
Agent safety	PASS
Security	No unresolved P0; P1 disposition approved
Performance	PASS on target hardware
Offline	PASS
Packaging	PASS
Backup/recovery	PASS
Documentation	COMPLETE

23. Recommended Test Execution Order

1. Unit validation → 2. integration/API → 3. UI → 4. AI benchmark → 5. RAG evaluation → 6. agent safety → 7. security → 8. performance → 9. offline → 10. packaging/install → 11. full regression → 12. release candidate → 13. final acceptance.

24. Final Testing, Security & Release Statement

NEXUS AI is ready for release only when the system is proven—not merely demonstrated—to be functional, secure, locally operable and recoverable. The release gate combines conventional software testing with AI-specific evaluation, RAG grounding, agent safety, offline verification, hardware-aware performance testing and installer validation.

25. Next Engineering Dependency

N63–N72 complete the quality and release foundation. With the previous architecture, AI, RAG, agent, UI and data documents, the project now has a complete product-definition and validation baseline. The next practical step is to convert these documents into the actual repository structure, implementation tasks, development sprints and build sequence.